

CS410

Parallel Computing: An Introduction

Milind Chabbi

Nikhil Hegde

Department of Computer Science and Engineering
IIT Dharwad

Slide adapted from John Mellor-Crummey's Comp422 @ Rice University

Application Pull

- **Complex problems require computation on large-scale data**
- **Sufficient performance is available only through massive parallelism**

Computing and Science

“Computational modeling and simulation are among the most significant developments in the practice of scientific inquiry in the 20th century. Within the last two decades, scientific computing has become an important contributor to all scientific disciplines. It is particularly important for the solution of research problems that are insoluble by traditional scientific theoretical and experimental approaches, hazardous to study in the laboratory, or time consuming or expensive to solve by traditional means”

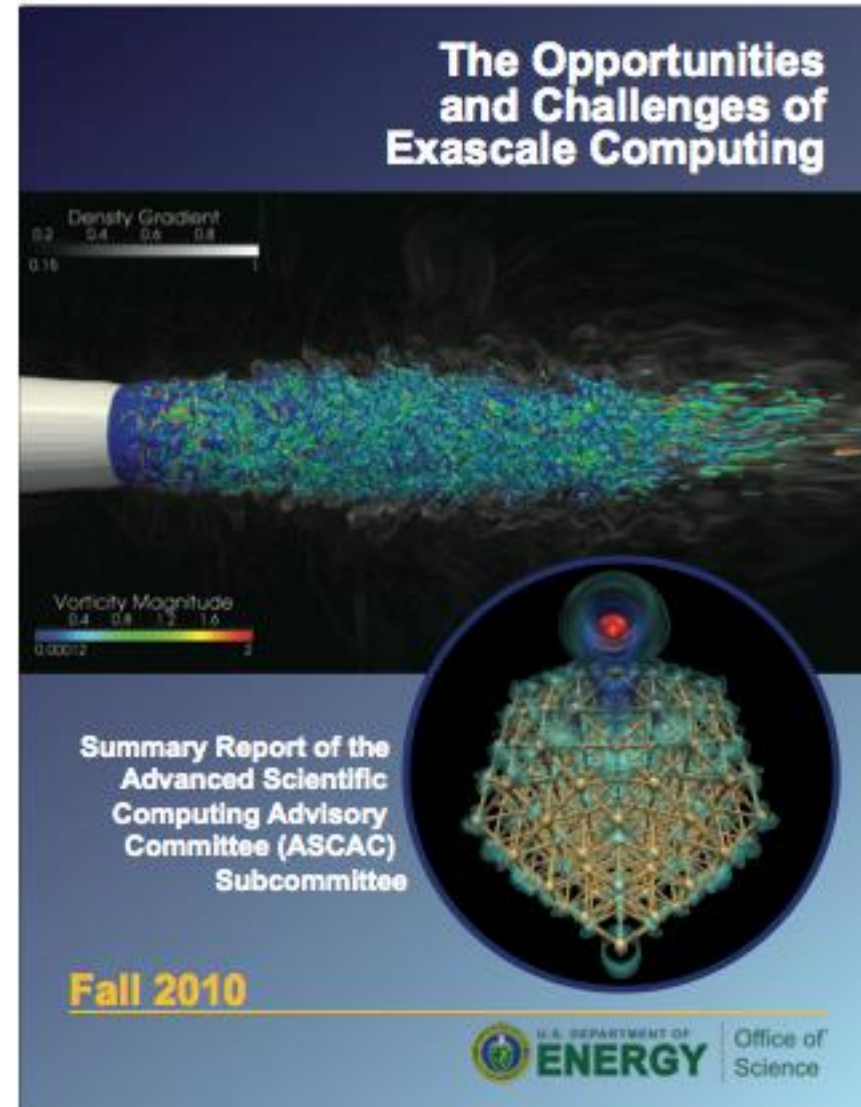
— **“Scientific Discovery through Advanced Computing”**
DOE Office of Science, 2000

The Need for Speed: Complex Problems

- **Science**
 - understanding matter from elementary particles to cosmology
 - storm forecasting and climate prediction
 - understanding biochemical processes of living organisms
- **Engineering**
 - multiscale simulations of metal additive manufacturing processes
 - understanding quantum properties of materials
 - earthquake and structural modeling
 - pollution modeling and remediation planning
 - molecular nanotechnology
- **Business**
 - computational finance - high frequency trading
 - information retrieval
 - data mining “big data”
- **Defense**
 - nuclear weapons stewardship
 - cryptology

The Scientific Case for Exascale Computing

- Predict regional climate changes: sea level rise, drought and flooding, and severe weather patterns
- Reduce carbon footprint of transportation
- Improve efficiency and safety of nuclear energy
- Improve design for cost-effective renewable energy resources such as batteries, catalysts, and biofuels
- Certify the U.S. nuclear stockpile
- Design advanced experimental facilities, such as accelerators, and magnetic and inertial confinement fusion
- Understand properties of fission and fusion reactions
- Reverse engineer the human brain
- Design advanced materials



Earthquake Simulation in Japan

Powerful earthquakes leave at least 57 dead, destroy buildings along Japan's western coast



Earthquake Simulation in Japan

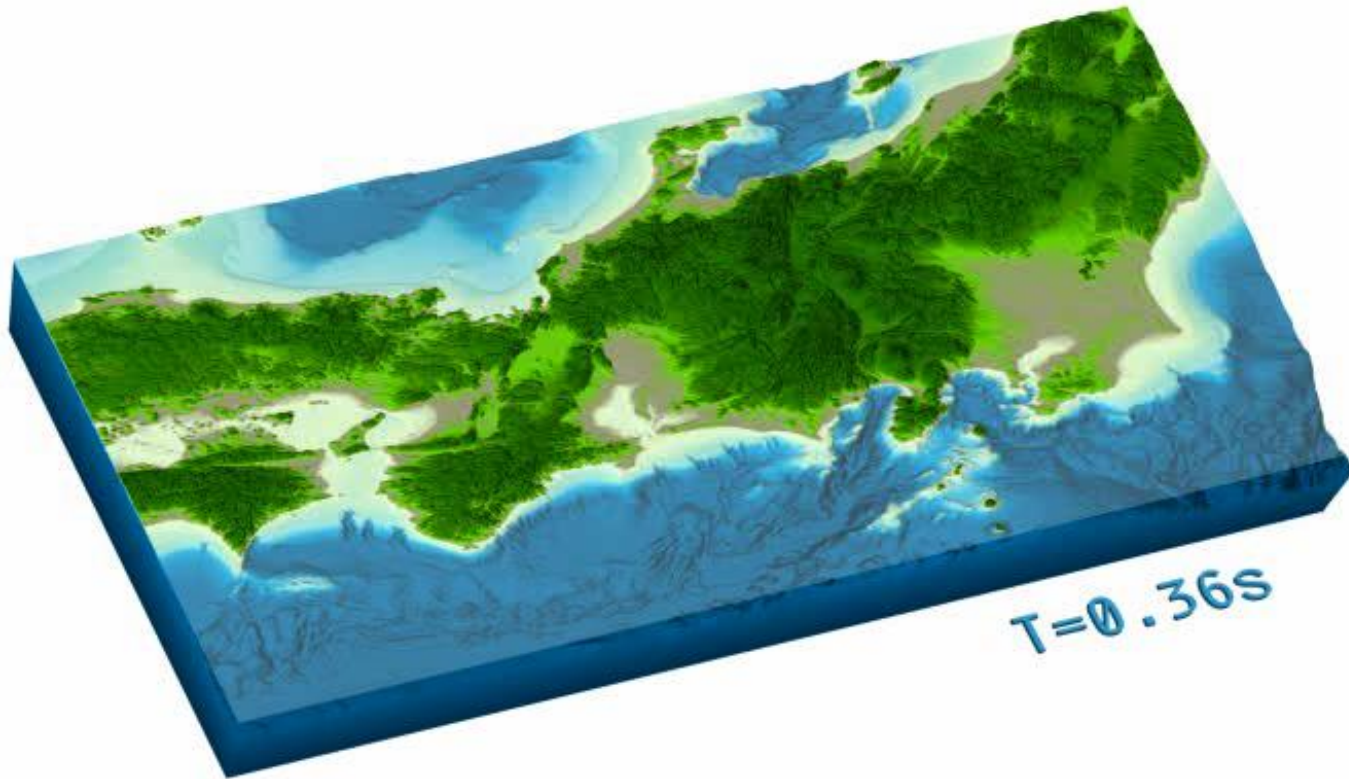
March 11, 2011 Fukushima Daiichi Nuclear Power Plant suffered major damage from a 9.0 earthquake and subsequent tsunami that hit Japan.

The earthquake and tsunami disabled the reactor cooling systems, leading to radiation leaks and triggering a 30 km evacuation zone around the plant.

Confirmed deaths: > 18,500 as of 2021

M7.5 Anamizu, Japan - EARTHQUAKE SIMULATION
<https://www.youtube.com/watch?v=NSGURtZSu-w>

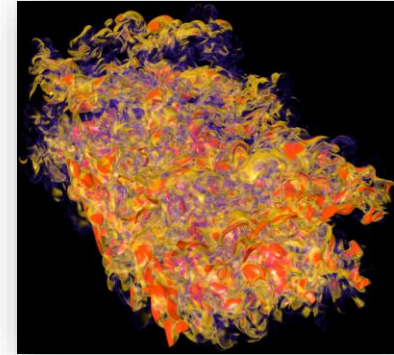
Earthquake Simulation in Japan



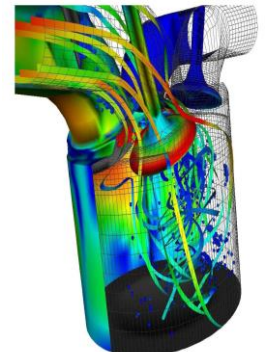
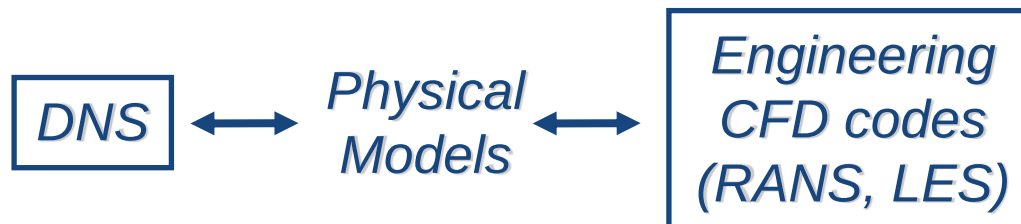
To watch this animation, check [earthquake_tsunami_wave_simulation.mov](#)

Simulating Turbulent Reacting Flows: S3D

- Direct numerical simulation (DNS) of turbulent combustion
 - state-of-the-art code developed at CRF/Sandia
 - PI: **Jaqueline H. Chen, SNL**
 - 2020: 600K hours, IBM AC922 2xP9+6xV100
 - “DNS of Turbulent Combustion Towards Efficient Engines with In Situ Analytics”



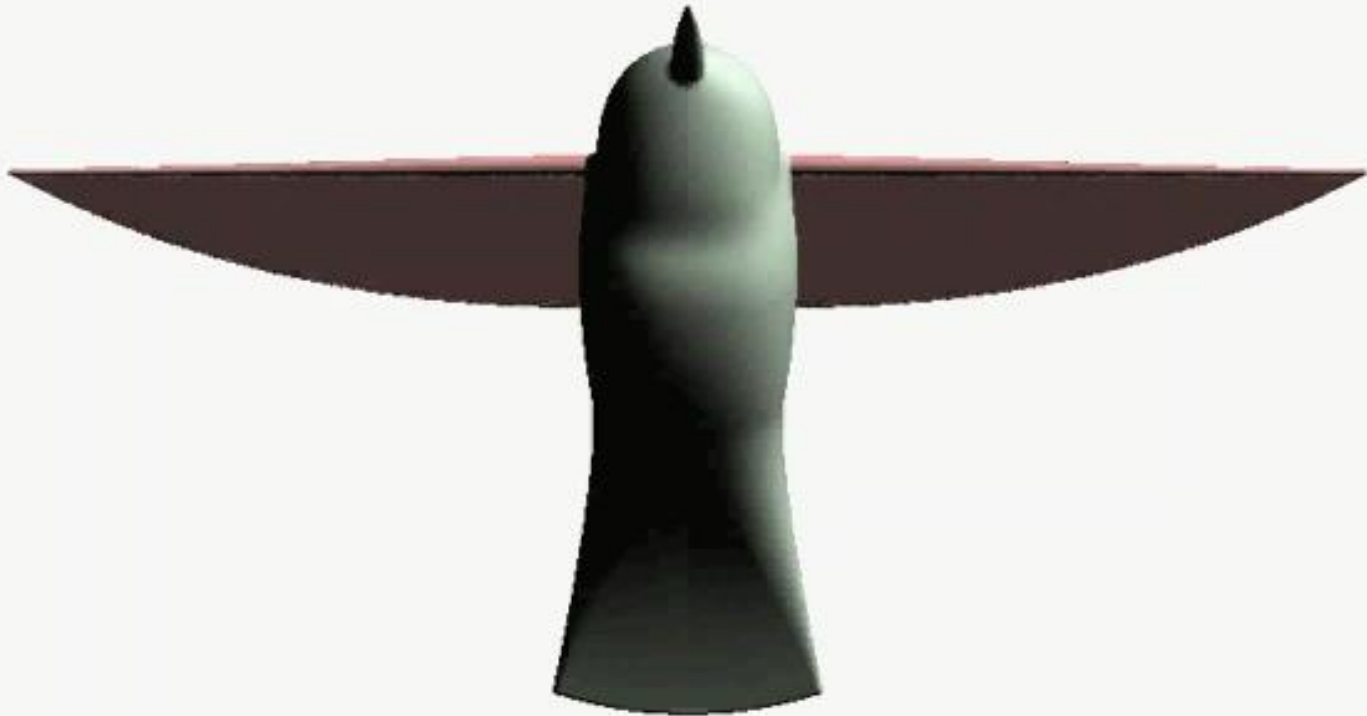
- Science
 - study micro-physics of turbulent reacting flows
 - physical insight into chemistry turbulence interactions
 - simulate chemistry and multi-physics (sprays, radiation, soot)
 - develop and validate reduced model descriptions used in macro-scale simulations of engineering-level systems



Fluid-Structure Interactions

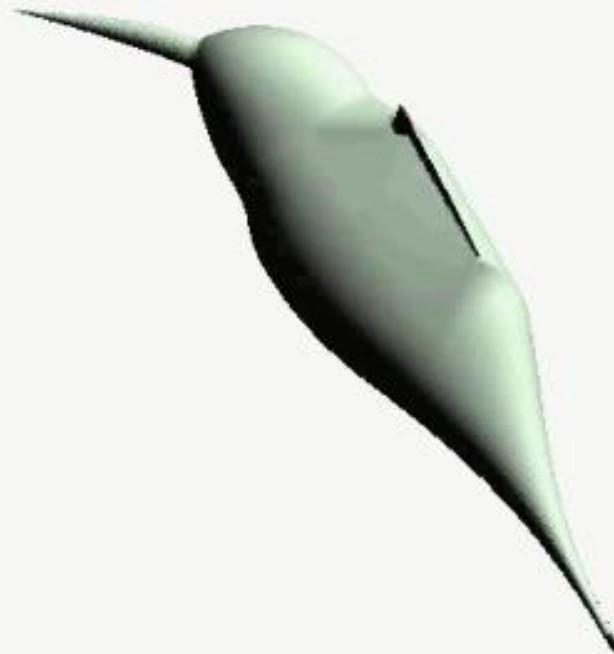
- **Simulate ...**
 - rotational geometries (e.g. engines, pumps), flapping wings
- **Traditionally, such simulations have used a fixed mesh**
 - drawback: solution quality is only as good as initial mesh
- **Dynamic mesh computational fluid dynamics**
 - integrate automatic mesh generation within parallel flow solver
 - nodes added in response to user-specified refinement criteria
 - nodes deleted when no longer needed
 - element connectivity changes to maintain minimum energy mesh
 - mesh changes continuously as geometry + solution changes
- **Example: 3D simulation of a hummingbird's flight**
 - [Andrew Johnson, AHPCRC 2005]

Air Velocity (Front)



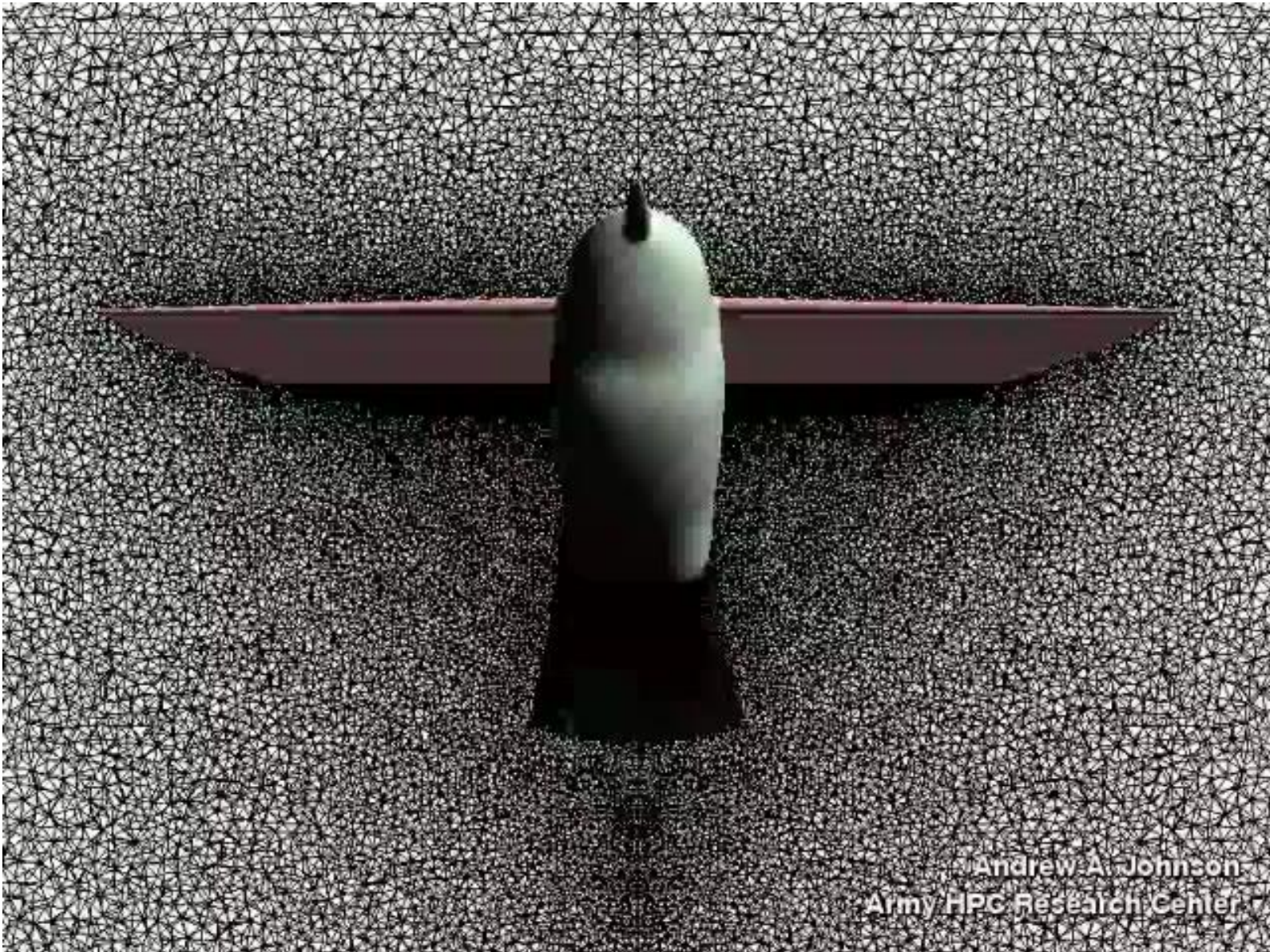
Andrew A. Johnson
Army HPC Research Center

Air Velocity (Side)

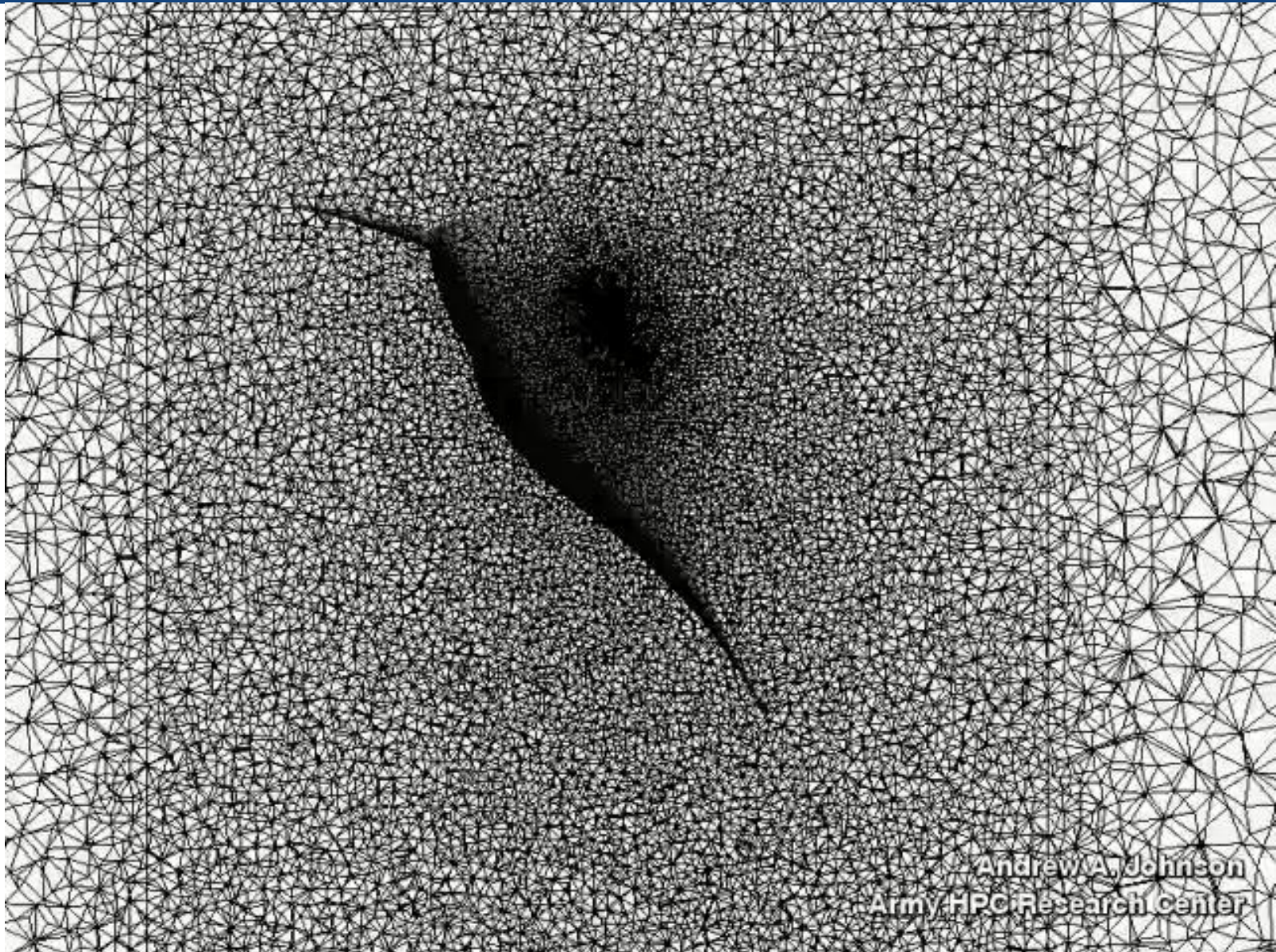


Andrew A. Johnson
Army HPC Research Center

Mesh Adaptation (front)



Mesh Adaptation (side)



3D Simulations

- hummingbird's flight
—<https://www.youtube.com/watch?v=Dg8xg4U7Xqs>
- 3D heart simulation
—<https://youtu.be/2LPboySOSvo>
- Airplane wing stall
—<https://www.youtube.com/watch?v=gjNPEDBeiTI>

Hummingbird Simulation in UPC

- **UPC: PGAS language for scalable parallel systems**
 - supports a shared memory programming model on a cluster
- **Application overview**
 - distribute mesh among the processors
 - partition the mesh among the processors
 - each processor maintains and controls its piece of the mesh
 - has a list of nodes, faces, and elements
 - communication and synchronization
 - read-from or write-to other processor's data elements as required
 - processors frequently synchronize using barriers
 - use “broadcast” and “reduction” patterns
 - constraint
 - only 1 processor may change the mesh at a time

Algorithm Sketch

At each time step...

- **Test if re-partitioning is required**
- **Set up interprocessor communication if mesh changed**
- **Split elements into independent (vectorizable) groups**
- **Calculate the refinement value at each mesh node**
- **Move the mesh**
- **Solve the coupled fluid-flow equation system**
- **Update the mesh to ensure mesh quality**
 - swap element faces to obtain a “Delaunay” mesh
 - add nodes to locations where there are not enough
 - delete nodes from locations where there are too many
 - swap element faces to obtain a “Delaunay” mesh

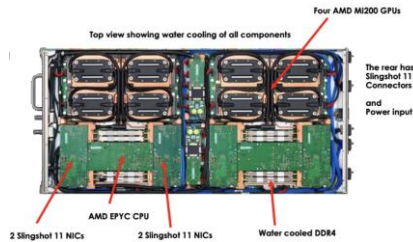
Challenges of Explicit Parallelism

- **Algorithm development is harder**
 - complexity of specifying and coordinating concurrent activities
- **Software development is much harder**
 - lack of standardized & effective development tools and programming models
 - subtle program errors: race conditions
- **Rapid pace of change in computer system architecture**
 - a great parallel algorithm for one machine may not be suitable for another
 - example: homogeneous multicore processors vs. GPUs

Parallel Hardware in the Large

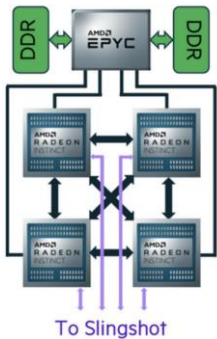
ORNL Frontier Supercomputer

Compute Blade (2 nodes)



Compute Node

AMD EPYC 64 core @ 2GHz
128 threads



4 x Instinct MI250X GPUs
14,080 stream processor each



Figure14: One Rack Containing 64 Blades (128 nodes)

System Characteristics	
Node	1 - AMD EPYC 7A53 CPU & 4 - AMD Instinct MI250X GPUs
Nodes per Blade	2
Nodes per Cabinet	128
Nodes per System	9408
Total system memory:	9.2 PB (4.6 PB HBMe2 + 4.6 PB DDR4)
Total on-node NVM	37 PB (66 TB/s read, 62 TB/s write)
Frontier Storage	706 PB (695 PB disk + 11 PB SSD (9.4 TB/s))
Memory Bandwidth between HBM2e and each GPU	3,200 GB/s (3.2 TB/s)
Memory Bandwidth between DDR4 and the CPU	205 GB/s

1.6 x 10¹⁸ operations/second = 1.6 ExaFlops



Compute System

El Capitan - Supercomputer @ LLNL

- <https://asc.llnl.gov/exascale/el-capitan>
- 7500sqft footprint, about 87 racks
- Built by HPE, Contains AMD MI300A APUs (Accelerator Processing Unit: CPU+GPU fused in a single chip)

Scale of the Largest HPC Systems (Nov 2024)

A	E	F	L	M	N	O	S
Rank	Name	Computer	Total Core	Accelerator/Co	Rmax [TFlop/	Rpeak [TFlop/s	Power (k
1	El Capitan	HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS	11,039,616	9,988,224	1,742,000.00	2,746,376.09	29,580.98
2	Frontier	HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray	9,066,176	8,451,520	1,353,000.00	2,055,716.99	24,607.00
3	Aurora	HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max,	9,264,128	8,159,232	1,012,000.00	1,980,006.00	38,698.36
4	Eagle	Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR	2,073,600	1,900,800	561,200.00	846,835.20	
5	HPC6	HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9	3,143,520	2,930,400	477,900.00	606,965.76	8,460.90
6	Supercomputer Fugaku	Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D	7,630,848		442,010.00	537,212.00	29,899.23
7	Alps	HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS	2,121,600	1,372,800	434,900.00	574,841.28	7,124.00
8	LUMI	HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11	2,752,704	2,566,080	379,700.00	531,505.15	7,106.82
9	Leonardo	BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100	1,824,768	1,714,176	241,200.00	306,311.16	7,493.74
10	Tuolumne	HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS	1,161,216	1,050,624	208,100.00	288,881.05	3,386.62

>1M cores

5 countries : USA
(5), Italy (2), Japan,
Switzerland, Finland

9 are
heterogeneous

Scale of the Largest HPC Systems (Nov 2023)

	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)
1	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,194.00	1,679.82
2	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	4,742,808	585.34	1,059.33
3	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Microsoft Azure United States	1,123,200	561.2	846.84
4	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21
5	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.7	531.51
6	Leonardo - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, EVIDEN EuroHPC/CINECA Italy	1,824,768	238.7	304.47
7	Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148.6	200.79
8	MareNostrum 5 ACC - BullSequana XH3000, Xeon Platinum 8460Y+ 40C 2.3GHz, NVIDIA H100 64GB, Infiniband NDR200, EVIDEN EuroHPC/BSC Spain	680,960	138.2	265.57
9	Eos NVIDIA DGX SuperPOD - NVIDIA DGX H100, Xeon Platinum 8480C 56C 3.8GHz, NVIDIA H100, Infiniband NDR400, Nvidia NVIDIA Corporation United States	485,888	121.4	188.65
10	Sierra - IBM Power System AC922, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	1,572,480	94.64	125.71

>400K cores

5 countries

9 are
heterogeneous

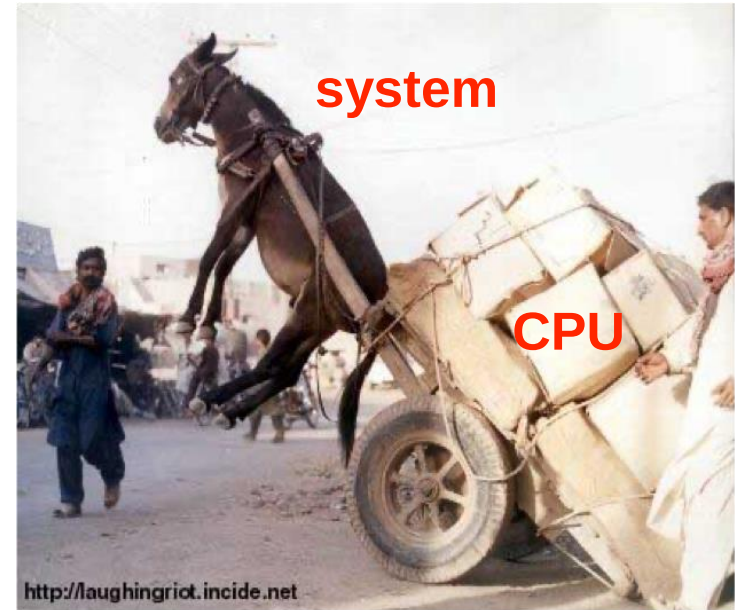
Parallel System for the Course

- **GPU Cluster @ IIT Dharwad**
 - Nvidia DGX100 – 8 Nvidia V100 cards
- **HPC Cluster @ IIT Dharwad**
 - Fujitsu, 30 Nodes, 90TF (peak)
- **CDAC Cluster (Param Utkarsh, Param Rudra)**

Achieving High Performance on Parallel Systems

Computation is only part of the picture

- **Memory latency and bandwidth**
 - CPU rates are $> 200\times$ faster than memory
 - bridge speed gap using memory hierarchy
 - more cores exacerbates demand
- **Interprocessor communication**
- **Input/output**
 - I/O bandwidth to disk typically needs to grow linearly with the # processors
 - increasing using SSD for acceleration



Challenges of Parallelism in the Large

- **Parallel science applications are often very sophisticated**
—e.g. adaptive algorithms may require dynamic load balancing
- **Multilevel parallelism is difficult to manage**
- **Extreme scale exacerbates inefficiencies**
—algorithmic scalability losses
—serialization and load imbalance
—communication or I/O bottlenecks
—insufficient or inefficient parallelization
- **Hard to achieve top performance even on individual nodes**
—contention for shared memory bandwidth
—memory hierarchy utilization on multicore processors

Next Class

- **Introduction to parallel algorithms**
 - tasks and decomposition
 - task dependences and critical path
 - mapping tasks
- **Decomposition techniques**
 - recursive decomposition
 - data decomposition